

TD3 ASD1

Rappel:

Définition :

Une **pile** est une structure de données **linéaire** où l'on insère et retire les éléments **du même côté** appelé **sommet (top)**.

Elle suit le principe :

L.I.F.O. = *Last In, First Out*
(*Le dernier entré est le premier sorti.*)

Opérations principales :

Opération	Description
Empiler(x)	Ajoute un élément x au sommet de la pile
Dépiler()	Retire l'élément au sommet
Sommet()	Retourne la valeur au sommet sans la retirer
PileVide()	Renvoie Vrai si la pile est vide
PilePleine()	Renvoie Vrai si la pile est pleine (dans une implémentation statique)

Exemple :

Si on empile successivement :

Empiler(5), Empiler(7), Empiler(9)

Pile : [5, 7, 9]

Puis Dépiler() → on retire 9

Pile devient [5, 7]

La File (Queue)

Définition :

Une **file** est une structure de données **linéaire** où :

- l'insertion se fait à une extrémité : **l'arrière (queue)**
- la suppression se fait à l'autre extrémité : **l'avant (tête)**

Elle suit le principe :

F.I.F.O. = *First In, First Out*

(*Le premier entré est le premier sorti.*)

Opérations principales :

Opération	Description
Enfiler(x)	Ajoute un élément x à la fin de la file
Defiler()	Retire l'élément au début de la file
Tete()	Retourne la valeur en tête sans la retirer
FileVide()	Renvoie Vrai si la file est vide

Exemple :

Si on enfile successivement :

`Enfiler(2), Enfiler(4), Enfiler(6)`

File : [2, 4, 6]

Puis `Defiler()` → on retire 2

File devient [4, 6]

Exercices d'Application :

Exercice 1 :

Écrire un algorithme qui permet :

1. D'empiler n entiers dans une pile.
2. D'afficher le contenu de la pile.
3. De dépiler tous les éléments en les affichant un par un.

Exercice 2 :

Écrire un algorithme qui lit un mot (ex. "SALUT"), empile ses lettres, puis dépile pour afficher le mot à l'envers ("TULAS").

Exercice 3 :

On simule une file d'attente dans une banque.

Chaque client arrive (enfile) et quitte le guichet (défile). Écrire un algorithme qui :

1. Enfile n clients avec leur numéro.
2. Défile les clients un à un en affichant leur ordre de passage.

Exercice 4

Écrire un algorithme qui vérifie si une expression mathématique contient des **parenthèses bien équilibrées**. Exemple :

- $(a+b) * (c+d)$ équilibrée
- $(a+b * (c-d)$ non équilibrée

Principe :

- Parcourir la chaîne caractère par caractère.
- Empiler quand on lit ' ('.
- Dépiler quand on lit ') '.
- Si la pile est vide à la fin $\rightarrow$ parenthèses équilibrées.